EXERCISES

1. Parking structure controller

Write properties that define the following requirements. A package and an interface are provided below.

/* Problem Statement[21]

The example FSM controls a FIFO Queue but to be practical, consider that we want to design a simple digital controller that monitors cars entering and leaving a parking lot. The operation specs are as follows.

* The controller maintains the current COUNT of the total number of cars parked in the lot.
* The lot has a limited number of parking spots, for simplicity sixteen (16).
* There are two input sensors at the entrance and exit of the lot. The sensors provide input signals to the controller as cars pass by and the controller increments/decrements the current COUNT. What if two cars enter and exit simultaneously?
* If COUNT=16, then the controller issues a FULL output signal, otherwise the OPEN output signal is on.
* There is a manual input for the lot attendant to override the COUNT for special vehicles.
* A START/STOP input is also available to the attendant, with the STOP input causing a CLOSED output signal.

*/

Figure 6.ex1 is a pictorial view of the parking lot.

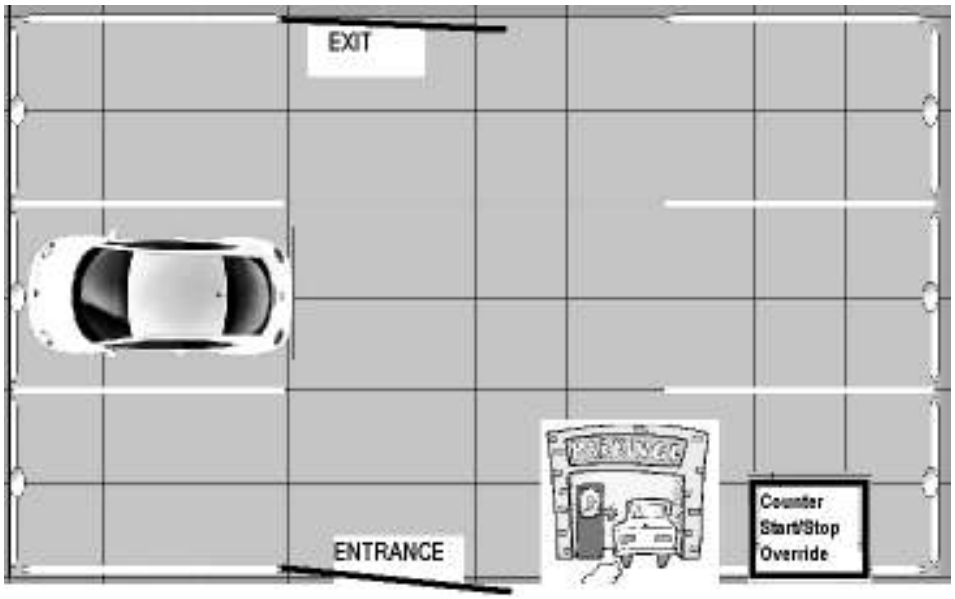

Figure 6.ex1 Parking Lot overview

[21] Requirements extracted from http://bear.ces.cwru.edu/eecs_316/
Instructors: Prof. Chris Papachristou, Francis Wolff

```
package parking_pkg;
  timeunit 1ns;
  timeprecision 100ps;
// Type defintions
  typedef enum bit[1:0] {NEUTRAL, STOP, START} gate_override_t;
  typedef enum bit {CLOSED, OPEN} gate_t;
  typedef enum bit {ENABLE_COUNTER, BYPASS_COUNTER} counter_cntrl_t;
endpackage : parking_pkg

interface parking_structure_if (input wire clk, reset_n);
  import parking_pkg:: *;
  bit           parking_full;
  bit           entrance_sensor;
  bit           exit_sensor;
  gate_override_t  gate_override;
  counter_cntrl_t  counter_cntrl;

  modport parking_cntlr_if (
  output gate_t          gate_status,
  output bit             parking_full,
  input  bit             entrance_sensor,
  input  bit             exit_sensor,
  input  gate_override_t  gate_override,
  input  counter_cntrl_t  counter_cntrl);
endinterface : parking_structure_if
```

2. Handshake Model

Write properties that define the following requirements for a slave unit that acquires a bus through a handshake protocol. The protocol requirements can be summarized as follows:

1. The slave issues a bus request (*req)*, and holds that request until master asserts the acknowledge (*ack)* signal, at which time the slave will deassert the bus request.
2. The master replies to bus request with a single acknowledge cycle, asserted within four cycles from the bus request.
3. In response to *ack* signal, the slave issues
    a. *Start* for 1 cycle, used for synchronization purposes by master.
    b. *Valid* for 4 cycles, used for data envelope. The *valid* signal occurs in the same cycle as the *start.* Data is not within the design of this slave.
    c. *Done* for 1 cycle, used for synchronization purposes by master. The *done* signal occurs 1 cycle after the last cycle of *valid.*
4. An active *reset* signal aborts any handshake operation.
5. All operations are synchronous to the system clock.

Figure 6.ex2 demonstrates a typical timing for the handshake protocol.

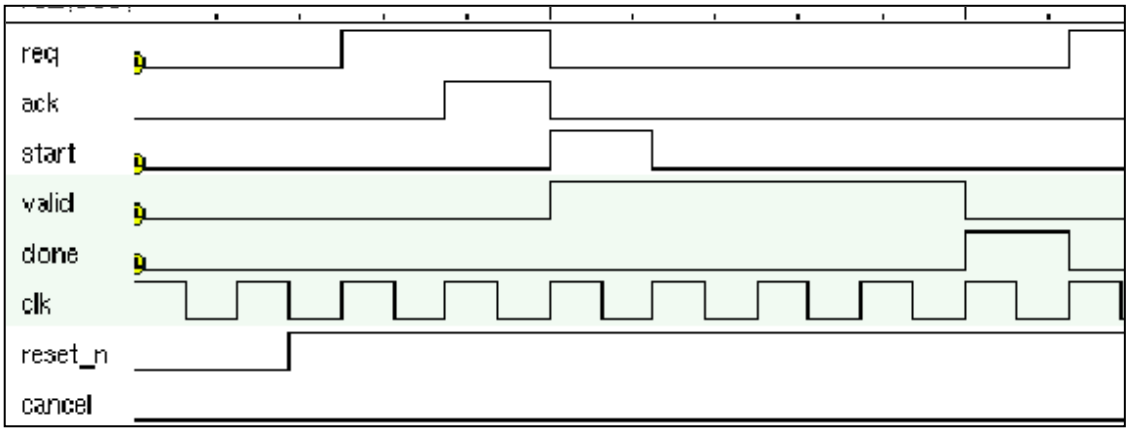

Figure 6.ex2 Handshake Protocol Timing Example